Data Storage for LLM

İnan Deniz Bal

Overall

Data Formats

Data Distribution

Data Organization

Data Movement

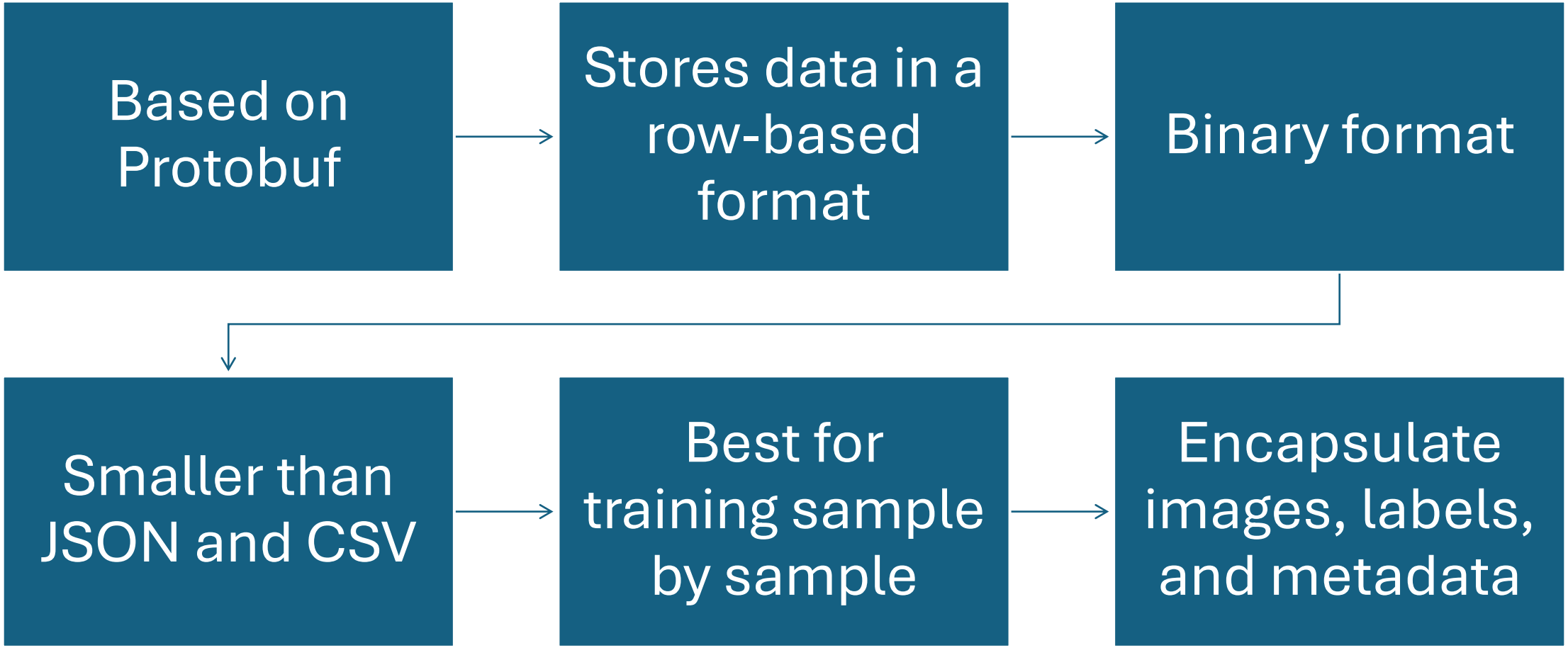
Data Fault Tolerance

KV Cache

1) Data Formats

- High read efficency under pressure = PROBLEM
- Current Tech => USE SEQUENTIAL
- CSV, JSON, TSV, TXT good for Pure-Text bad for PB scale
- Lack of compression = Incur significant storage overhead
- Not good for Multi-Model formats

TFRecord


Based on Protobuf
Stores data in a row-based format
Binary format
Smaller than JSON and CSV
Best for training sample by sample
Encapsulate images, labels, and metadata

Tensor Data Formats

Multi-dimensional arrays

Partitioned and processed in parallel

Dense multi-dimensional storage

Large storage overhead

Poor readability

Model Data Format

- Means stored LLM file

Format	Purpose	Advantage	Disadvantage / Note
Pickle (.pkl)	General Python object serialization	Convenient, can store any Python object	Serious security risk (arbitrary code execution)
Safetensors	Secure and fast storage of model weights	High security, fast loading (mmap)	Stores only tensors (weights)
ONNX	Cross-framework portability	Interoperability across different frameworks	Does not store training state
SavedModel	Storing the entire model in TensorFlow	Includes the computation graph	Framework-specific

2) Data Distribution

Single node cannot store such large-scale data

Distributed across multiple nodes

Key Technologies

- Distributed Storage Systems
- Heterogeneous Storage Systems

Distributed Storage Systems

- Traditional like HDFS = High Cost
- Most use POSIX = Cost
- JuiceFS = Object Storage + Horizontal = -%20 Cost
- CRAQ algorithm for consistency
  - Data saved to multiple same chunks (form a chain)
  - For read any chunk from chain
  - For write sequentially on each chunk
  - Error in chunk = moves error chunk at the end
  - No impact on read so good for LLM

Heterogeneous Storage Systems

Deploying the model state across diverse storage media (GPUs, CPUs, NVMes Memory)

Distributing over GPU = More computational cost

Allows much larger models to be trained using the same computing resource (GPU) because not all of the memory has to reside on the GPU

vDDN & ZeRO-Infinity

- Per-layer memory management
- Sliding window that dynamically allocates memory
- Static policy: offloads feature maps of all layers or only convolutional layer
- Dynamic policy: determines which layers and convolutional algorithms to offload at runtime
- Fully utilizes CPU memory

vDDN & ZeRO-Infinity

Offloads model states to CPU

Memory-centric tiling technique : lowers working memory requirements

Utilize CPU's memory without leveraging its computational capabilities => harnessing the CPU's computational power

ProTrain

Doesn’t relly on manual parameter tuning (like vDDN and ZeRO)

Memory-Aware Runtime Profiler for monitoring

Persistent and non-persistent chunks based on usage patterns

Reduces redundant data

3) Data Organization

Lack of timeliness

Hallucination

RAG to address these limitations

Based on vector-based & graph-based organization

Vector-Based Organization

Converting data into vector form for efficient retrieval

Content Organization

- Refines source data into logical units

Chunking

- Divides long texts into appropriately sized segments

Embedding

- This process transforms the original input (text, images, audio) into dense vector representations

Compression

- Applying dimensionality reduction techniques

Storage

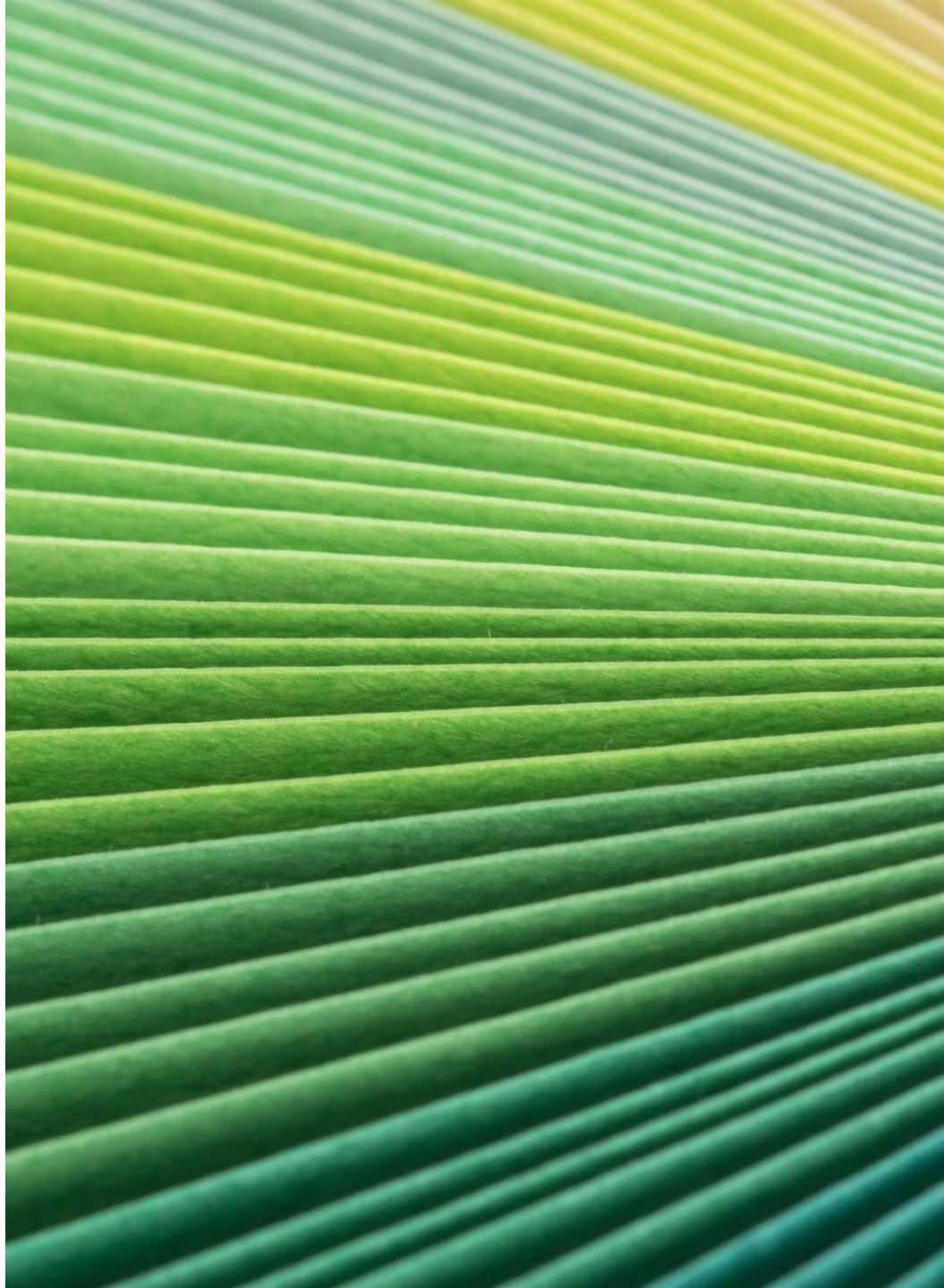
- Vector database

Graph-Based Organization

Explicitly represents entities and their relationships

Indexing

- GraphRAG
  - Leiden algorithm => detect tightly connected subgraphs
  - Generates hierarchical summaries each community
  - Triple is retrieved, the index sends them for inference
- LightRAG
  - Integrates deduplication functionality to identify and merge identical entities
- MiniRAG
  - Proposes a semantic-aware heterogeneous graph indexing mechanism,



4) Data Movement

Process of moving data from storage nodes to computing nodes

The main challenge is how to accelerate the data moving rate.

Data caching

Compute-storage overlap

Data/operator offloading

Data Caching

Aims to significantly **increase** the speed of data access

Capacity Waste: the fixed cache capacity >> what is actually needed

Poor Responsiveness: A fixed cache cannot adapt to the real-time,

Data Caching

Fluid: Dynamically adjusts cache capacity based on real-time I/O conditions and training speed using a **trial-and-error approach** inspired by TCP congestion control. It increases capacity when training speed increases and decreases it when speed falls

Quiver: Optimizes cache sharing by exploiting job I/O characteristics like **data shareability**, **substitutability** (I/O order doesn't affect correctness), and **predictability** (estimating I/O sensitivity) to improve performance even with small caches.

Tectonic-Shift: A hybrid storage architecture that integrates flash memory with traditional HDD systems. It dynamically manages segment admission and reinsertion into the flash cache by comparing **bucket priorities** (based on access patterns) against adjusted thresholds.

5) Data Fault Tolerance

- Quickly resume from the point of interruption
- Checkpoints: good for backup bad for compatibility on different
- PaddleNLP : model weights, optimizer weights, and other data in a unified safetensors format,
- Redundant Computations :
  - Inspired by the RAID disk redundancy technology
  - Each node computations both on the neural network layers and on some layers of its neighboring nodes as redundant computations.

6) KV Cache

- Avoids redundant computation by reusing stored key-value pairs improving efficiency
- Cache Space Management: separating the logical structure from its physical storage
- KV Placement : frequently used KV in faster storage media
- KV Shrinking: trimming or reducing the KV Cache to lower memory usage
- KV Indexing: Constructing an indexing architecture for the KV Cache to accelerate the query

Questions